

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I B Sandhya(192311292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sandhya

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Uneven Input Splits and Idle Reducers

Problem Analysis

The Hadoop job repeatedly fails because the input data is divided into **uneven input splits**, causing some Mapper tasks to receive much more data than others. As a result, some Mappers finish quickly while others

take much longer. Reducers may remain idle while waiting for the slow Mapper tasks to complete. This creates **data skew and poor resource utilization**.

Likely Design Error

The main design error is **improper input partitioning**. Very large or unbalanced files, inappropriate split sizes, or uneven data distribution can cause some tasks to process significantly more data than others. Poor partitioning of intermediate keys can also overload particular reducers.

Corrective Actions

- Use appropriate HDFS block and input-split sizes.
- Store data in reasonably balanced files.
- Use a suitable **InputFormat** to divide data efficiently.
- Use a **Combiner** to reduce intermediate data before the shuffle phase.
- Use an appropriate **Partitioner** to distribute keys evenly among reducers.
- Identify and handle data-skewed keys.
- Tune the number of Mapper and Reducer tasks according to available cluster resources.

Expected Result

Balanced input splits allow Mapper tasks to finish at approximately similar times. Proper partitioning distributes intermediate data more evenly among reducers, improving cluster utilization and reducing job execution time.

Conclusion

The failure is mainly caused by **uneven data partitioning and poor task distribution**. Correct split sizing, balanced files, combiners, and proper partitioning can prevent reducer idling and improve MapReduce performance.

2. HDFS Data Unavailability After a Single Node Failure

Problem Analysis

HDFS is designed to provide fault tolerance by maintaining multiple replicas of each data block. If data becomes unavailable after only one DataNode fails, the deployment probably has **insufficient replication or an incorrectly designed NameNode architecture**.

Likely Design Error

A possible error is setting the **replication factor to 1**, which means only one copy of a block exists. When that DataNode fails, the block becomes unavailable.

Another possibility is incorrect **NameNode configuration**. If there is only one NameNode and it fails, the HDFS namespace becomes unavailable even if the DataNodes are healthy.

Corrective Actions

- Set an appropriate HDFS replication factor, commonly **3** for important data.
- Use **rack-aware replication** so replicas are distributed across different racks.
- Deploy **NameNode High Availability** with Active and Standby NameNodes.

- Enable automatic failover where supported.
- Monitor DataNode health using heartbeats and block reports.
- Allow HDFS to automatically recreate missing replicas on healthy DataNodes.
- Regularly check the HDFS filesystem health and under-replicated blocks.

Expected Result

If one DataNode fails, another DataNode containing a replica can continue serving the requested data. NameNode HA also prevents the NameNode from becoming a single point of failure.

Conclusion

The unavailability is most likely caused by **insufficient block replication or lack of NameNode High Availability**. Replication, rack awareness, and redundant NameNodes provide better fault tolerance and availability.

3. Duplicate Records in ETL and Apache NiFi

Problem Analysis

The ingestion workflow using **ETL and Apache NiFi** produces duplicate records in the analytics store. This generally happens when the pipeline does not properly track whether a record has already been processed.

Likely Causes

Several factors may cause duplication:

- NiFi flow may process the same file more than once.
- There may be no unique identifier for records.
- Database extraction may repeatedly retrieve previously processed rows.
- A failed transaction may be retried without checking whether the previous attempt succeeded.
- Multiple ingestion flows may process the same source.
- The destination database may not enforce uniqueness.
- Checkpoint or state-management information may be missing or incorrectly configured.

Corrective Actions

- Use **unique record IDs** or business keys.
- Configure NiFi state management to track processed data.
- Use incremental extraction or **Change Data Capture (CDC)** for databases.
- Use checksums or hashes to identify duplicate files.
- Apply deduplication logic before loading the analytics store.
- Use database **primary keys or unique constraints**.
- Design the pipeline to support **idempotent processing**, meaning reprocessing the same input does not create another copy.
- Maintain proper failure and retry handling.

Example

Suppose a customer transaction has ID **TX1001**. If the same transaction reaches the analytics database twice, the unique transaction ID can be checked before insertion. The second occurrence can be ignored or updated instead of creating another record.

Conclusion

The duplication problem is mainly caused by **lack of state tracking, unique identifiers, and idempotent processing**. Using NiFi state management, CDC, deduplication, unique keys, and controlled retry mechanisms can prevent duplicate records.

4. YARN Resource Starvation

Problem Analysis

The YARN cluster experiences resource starvation when several users submit jobs at the same time. This indicates that resources such as **CPU and memory are not being distributed fairly** among applications or users.

Likely Scheduling Error

The cluster may be using an unsuitable scheduling policy or may not have properly configured **YARN queues and resource limits**. A large job could consume most of the available CPU and memory, leaving insufficient resources for other users.

For example, if one machine-learning job requests most of the cluster memory, smaller reporting and ETL jobs may remain waiting for resources.

Corrective Actions

- Configure **Capacity Scheduler or Fair Scheduler**.
- Create separate queues for different departments or workloads.
- Assign minimum and maximum capacity to each queue.
- Set user and application resource limits.
- Give important jobs appropriate scheduling priority.
- Prevent a single user from consuming all cluster resources.
- Monitor CPU, memory, queue utilization, and application wait time.
- Tune container sizes according to actual application requirements.

Suggested Queue Design

YARN Cluster

- **ETL Queue** – High priority
- **Reporting Queue** – Guaranteed capacity
- **ML Queue** – Controlled resource allocation

This allows important workloads to run without completely starving other applications.

Expected Result

With proper queue management, resources are shared more fairly. Large jobs cannot monopolize the entire cluster, and multiple users can run their workloads simultaneously.

Conclusion

The resource starvation is caused by **poor YARN scheduling and resource allocation**. A queue-based Capacity/Fair scheduling approach with resource limits and monitoring can provide fair and efficient cluster utilization.

5. Outdated Data After Backup Recovery

Problem Analysis

The backup system restores outdated data after a failure. This indicates that the organization is relying on an **old or improperly synchronized backup copy** instead of the latest valid data.

Likely Design Flaw

One possible error is confusing **HDFS replication with backup**. HDFS replication protects against DataNode failures by maintaining multiple copies, but it does not automatically provide historical backup versions.

Another problem may be that backups are performed too infrequently. If the last backup was created several hours or days before the failure, restoring it will cause recent updates to be lost.

Corrective Actions

- Schedule regular incremental and full backups.
- Maintain backup timestamps and version information.
- Use suitable backup and disaster-recovery tools.
- Maintain multiple backup versions according to the required retention period.
- Verify that backups are synchronized successfully.
- Use checksums or integrity verification.
- Periodically perform test restores.
- Keep a separate disaster-recovery copy so that failure of the main cluster does not destroy all copies.

Example

If the latest backup was taken at **10:00 AM** and the system fails at **5:00 PM**, restoring only the 10:00 AM backup loses seven hours of changes. A better strategy would combine periodic full backups with frequent incremental backups or another suitable change-based recovery mechanism.

Replication vs Backup

HDFS Replication	Backup
Protects against node/block failure	Protects against data loss and historical recovery
Maintains multiple current copies	Maintains recoverable versions
Usually within the storage cluster	Can be stored in a separate location
Does not replace backup	Provides disaster recovery and point-in-time recovery

Conclusion

The outdated recovery is likely caused by **infrequent backups, poor synchronization, or treating replication as a complete backup strategy**. Regular versioned backups, incremental backup mechanisms, integrity checks, and tested recovery procedures ensure that the latest valid data can be restored.